

Exercise 3: Train Route Planning [50 points]

Task 1: Specification

Let $Paths(G)$ be the set of all paths in G , $isPath(P, G) = P \in Paths(G)$, $pThrough(P, v) = v \in P$ and $L(P)$ denote the length of path P .

Define $Paths(G, start, target1, target2) =$

$$\{P \in Paths(G) : pThrough(P, start) \wedge pThrough(P, target1) \wedge pThrough(P, target2)\}, \quad (1)$$

and $L = \{L(P) : P \in Paths(G, start, target1, target2)\}$.

Then, $convvinientPath(graph, start, target1, target2) = P$ such that

$$P \in Paths(G, start, target1, target2) \wedge L(P) = \min(L). \quad (2)$$

Task 2: Algorithm

The **general idea** of our algorithm is as following:

- We create a minimal priority queue `minPQ` as a collection that stores all paths starting at `start` and passing through `target1` and `target2`, with key of each possible path being defined as its length.
- Next, starting from `start`, we conduct a DFS algorithm to search all paths passing through `target1` and `target2`.
- Once we find a path that passing through `target1` and `target2`, we push it into the `minPQ` declared above.
- Finally, we only need to pop the top element of `minPQ`

Task 3: Correctness

As what we have seen on the lectures, the algorithm involves the implementation of **DFS searching** and the methods of **minimal priority queue**.

Task 4: Complexity

Let N be the number of vertices in the graph

1. DFS searching for all paths that starting at `start`:

Worst-case time complexity: $O(N!)$

2. Insert a possible path into `minPQ`:

Worst-case time complexity: $O(\log N!) = O(N \cdot \log N)$

3. Overall worst-case time complexity:

$$O(N!) + O(N! \cdot \log N!) = O(N! \cdot \log N!) = O(N! \cdot N \cdot \log N) \quad (3)$$

Task 5: Java Implementation

See folder `ex03-code`.